

Introduction to the Bash shell

Alyssa Dai (she/her)
QLSC 612 - 2026 May 21

Lecture adapted from Brent McPherson, Sebastian Urchs, and [this](#) Software Carpentries course

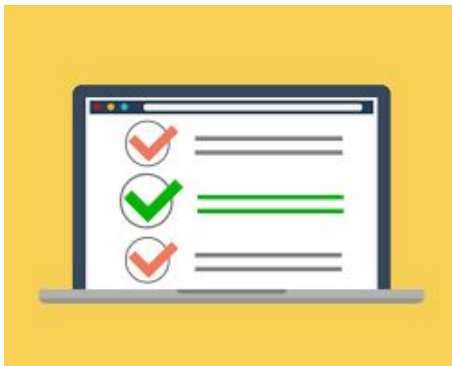


Before we get started

The slides can be found here (or in your local `QLS-course-materials` directory):
<https://github.com/neurodatascience/QLS-course-materials/tree/main/Lectures/2026>

To get the examples we'll use in the lecture, run the following commands in a **terminal**:

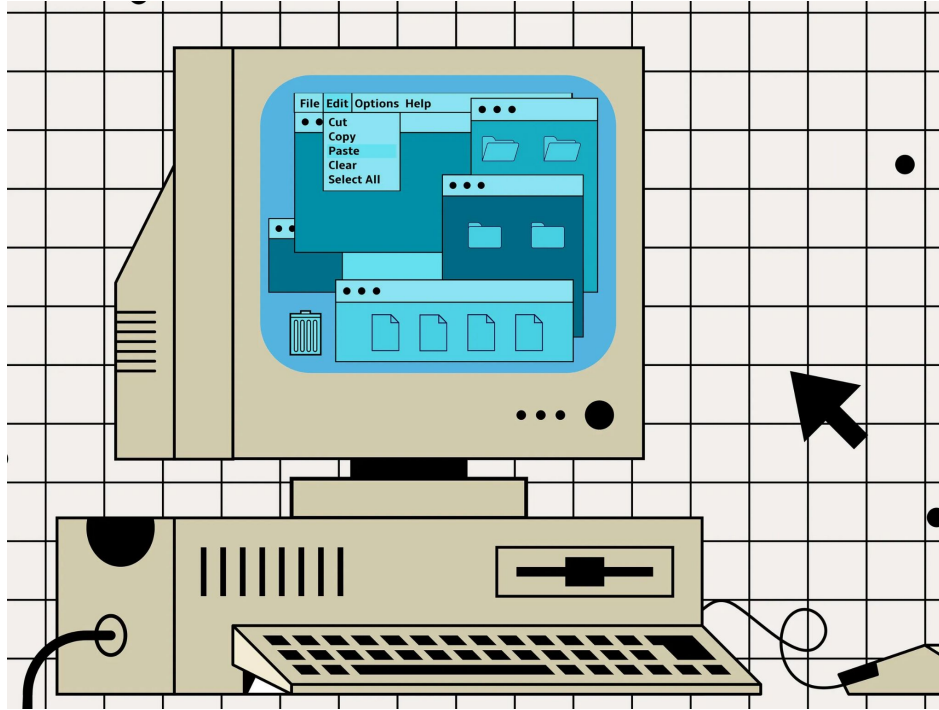
```
cp -r ~/QLS-course-materials/Lectures/2026/data/shell-course ~  
cd ~
```



Topics

- Basic definitions: what is a shell?
- Getting started in the shell
- Navigating the filesystem
- Working with files and directories
- Finding things using the shell
- Scripting basics
- *(if time)* Brief intro to High Performance Computing (HPC) clusters

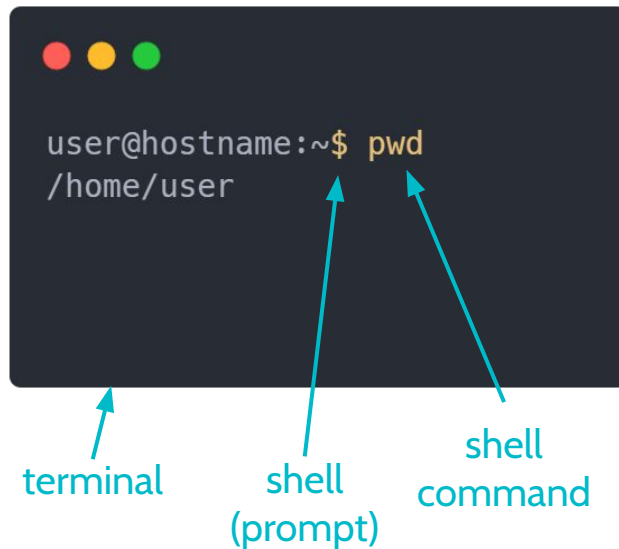
How do we interact with a computer?



You are currently using a graphical user interface (GUI)!

The command line

- Interaction with a computer through text commands
- A **shell** is a program that interprets user commands into something a computer can understand
- A shell runs inside a **terminal** that lets you type lines of commands
 - So, it provides a **command-line interface (CLI)**
 - This is in contrast to a **graphical user interface (GUI)**
- Command-line shells often provide a scripting language for automating sequences of commands



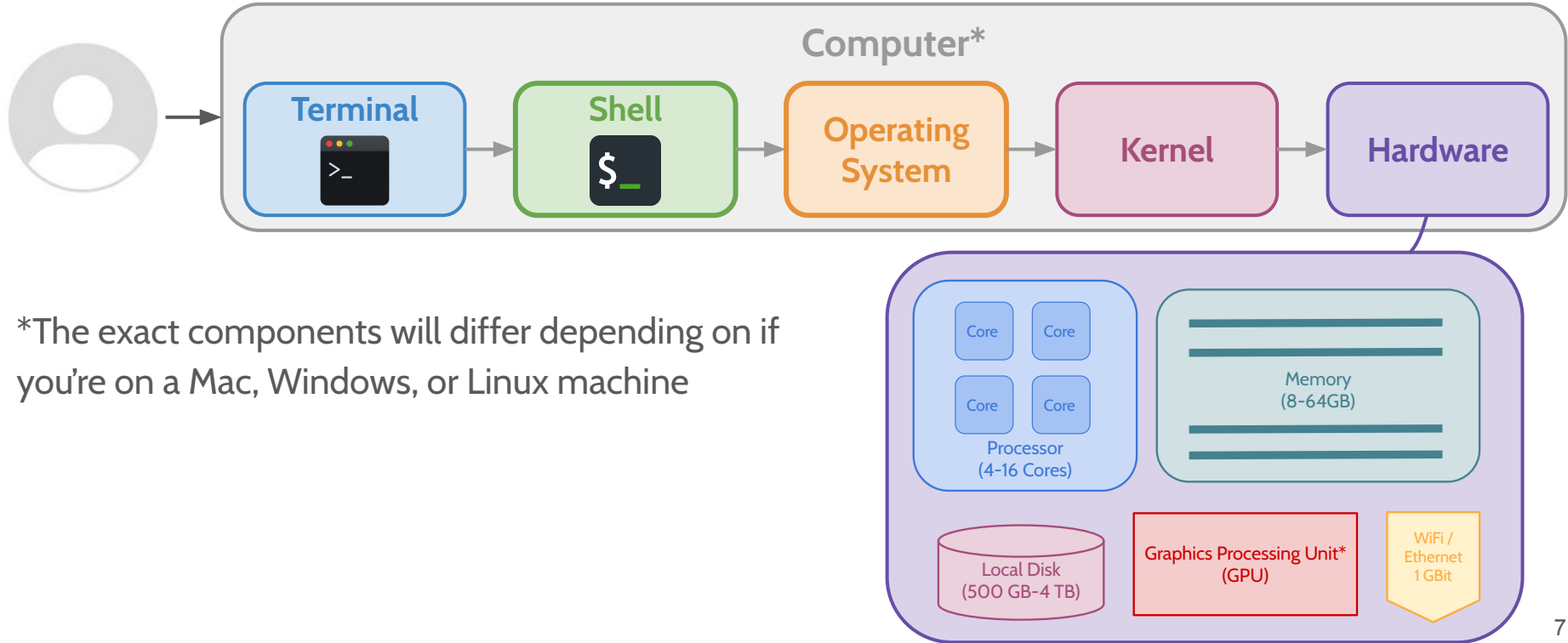
Where it all fits: a computer system

- Terminal
 - Program that provides access to an interactive shell
- Shell
 - Interactive program that interprets and runs commands
 - e.g., `bash`, `zsh`, `powershell`
- Operating System (OS)
 - Underlying software that coordinates program execution and provides a usable environment for interacting with a computer
 - e.g., Ubuntu, macOS, Windows
- Kernel
 - Core, low-level part of the OS that manages hardware and system resources
 - e.g., Linux kernel, XNU, Windows NT

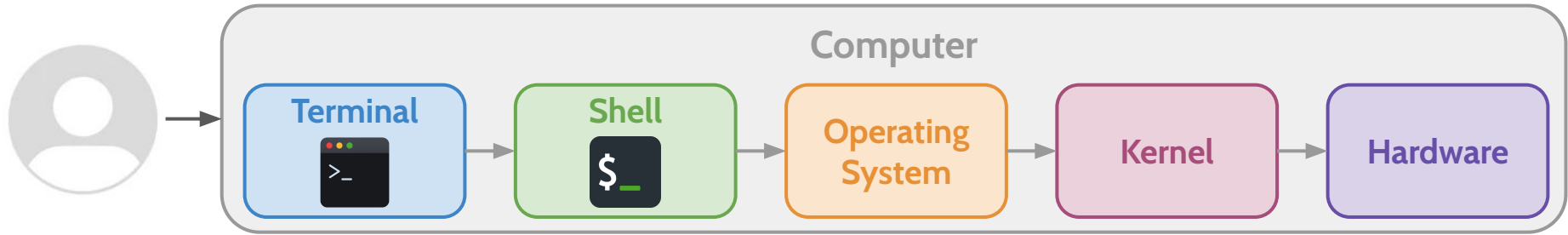


Distribution = a Linux-specific “flavour” of an OS

Where it all fits: a computer system

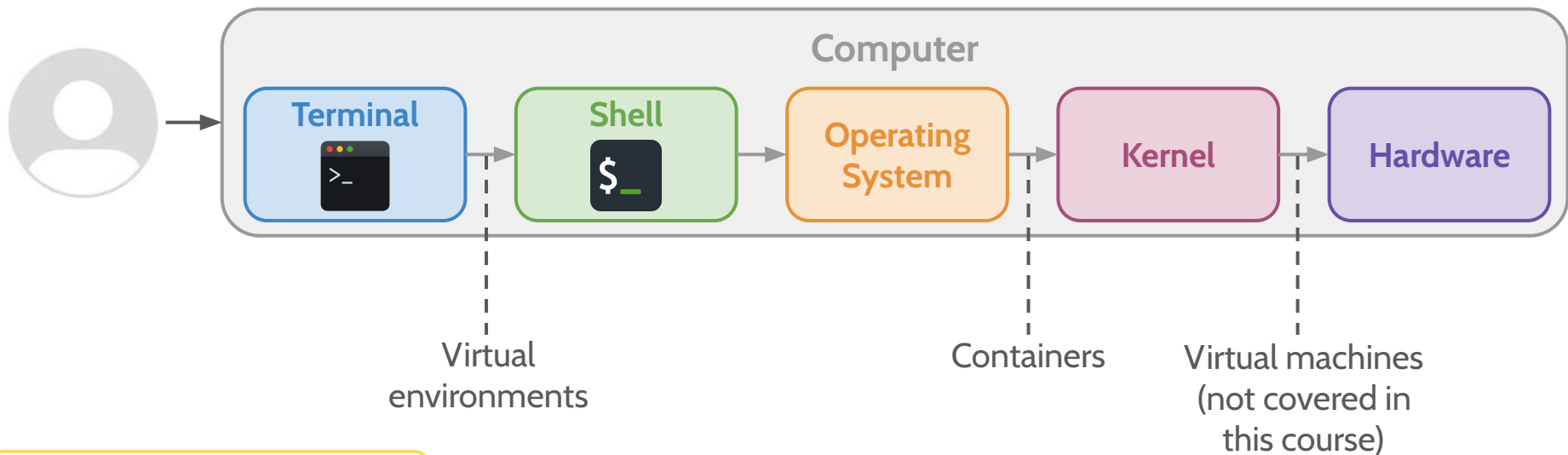


Where it all fits: a computer system



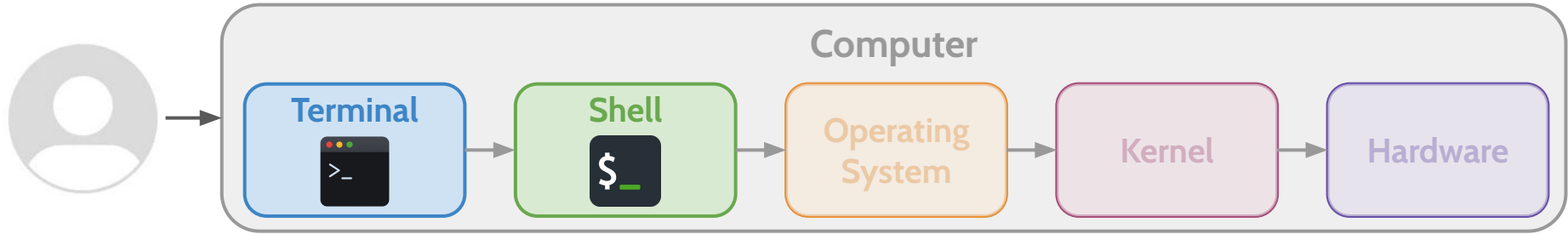
The shell provides direct access to OS functionality and low-level system operations

Software can be isolated for more reproducible results



💡 A preview for later modules!

Today's focus



There are many kinds of shells

- The most common shell is the **Bourne Again Shell** (`bash`)
- Some shells emphasize more human-readable syntax
 - e.g., **Friendly, Interactive Shell** (`fish`)
- Some shells emphasize richer interactive features and customization
 - e.g., **Z shell** (`zsh`), **Tenex C shell** (`tcsh`)
- Some shells emphasize more powerful scripting features
 - e.g., `bash`, `powershell` (Windows-focused)

What's the point of all the shells?

- They all have different strengths and weaknesses for different purposes
- You will see different shells preferred in different neuroimaging software
 - `bash` is used in FSL
 - `tcsh` is commonly used in AFNI and FreeSurfer



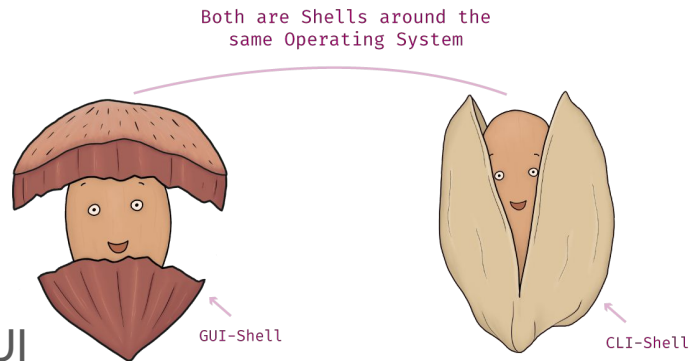
Today's focus: Bash



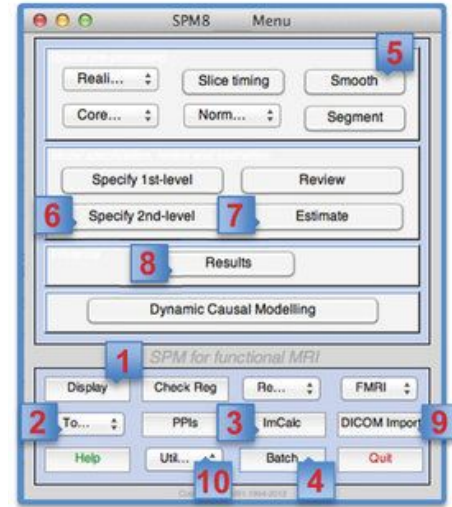
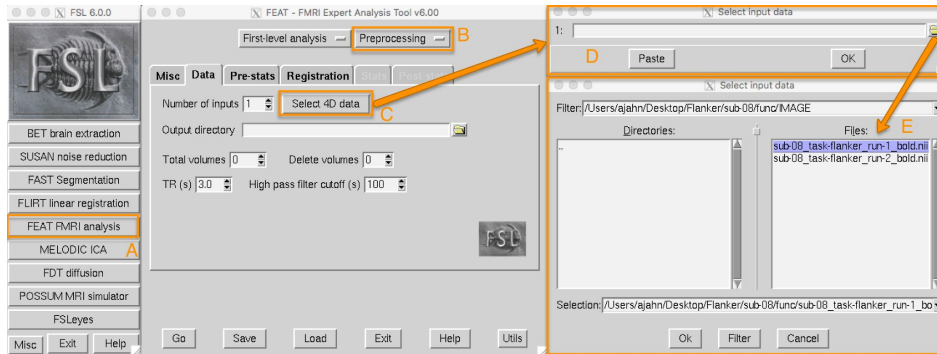
- The most ubiquitous shell environment
- Available for almost every OS
 - Default shell on most Linux systems
 - Compute Canada/most High Performance Computing systems (HPCs)
 - Default on Windows Subsystem for Linux (WSL)
 - macOS defaults to `zsh`, but `bash` is still available
- Shares similar command syntax with many common shells
- **Widely used in scientific computing and neuroimaging workflows**

But why use the shell at all?

- It's more powerful than a GUI!
- You can **automate** tasks that take many “clicks” in a GUI
- You can **sequence** together the output of one program into another (pipelines)
 - Flexible way to automate several tasks involving different tools
- Shell commands can be stored in scripts that can be **(re-)executed easily**
 - Self-documenting
 - Easily reproducible
- Essential for interacting with HPC environments and other remote machines
 - Easiest way to perform computational tasks on large volumes of data



Using a GUI is “easy”...



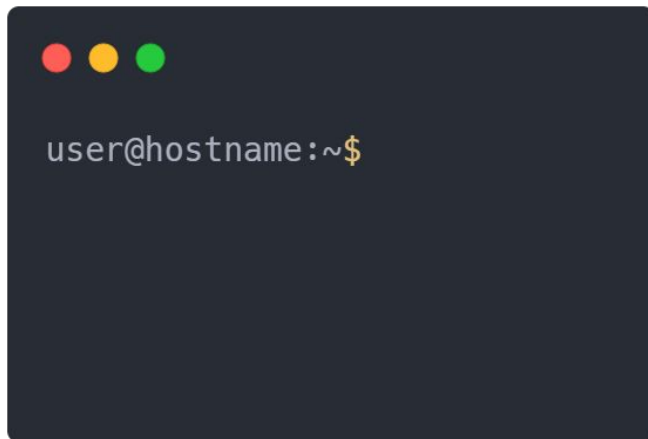
...but reproducible science is hard!

Getting started in the shell

Let's open up a terminal

The shell prompt

- When the terminal is first opened, you are presented with a **prompt**: `$`
 - This indicates the shell is waiting for input
 - Can be a different symbol other than `$`



What shell am I using?

- Check by typing: `echo $0`
 - `echo` is a command that means “print to screen”
- It should display `bash`
- If not, you can explicitly enter a Bash session by typing: `bash`
 - A shell can launch another interactive shell session (`bash`, `zsh`, etc.)
 - This is a *subprocess*



This is ≠ the `$` denoting the shell prompt

What does `$0` mean?

- The prefix `$` is used in bash to reference a **variable**
 - Specifically, the **value** of a variable
 - This can be a variable set by you or by the system
- By default, your shell will have some variables already available. These are **environment variables**. e.g.,
 - `$SHELL` : the path to your default shell (e.g., `/bin/bash`)
 - `$0` : the name of the current running program (in this case, your active shell)
 - `$USER` : the current username
- We will learn more about variables shortly!

Helpful shell shortcuts

- Tab completion: **Tab** to fill in available completions for your current prompt
- Command history: **Up** to cycle through previously entered commands
- There are also many keyboard shortcuts with **CTRL**
 - **CTRL+C** : end the current running command/process (but not the session)
 - **CTRL+Left** or **CTRL+Right** : move cursor one word to the left or right
 - **ALT+Backspace** / **Ctrl+W**: delete entire previous “word”
 - **CTRL+A** and **CTRL-E** : go to the beginning / end of the current line
 - **CTRL+R** : search the command history
 - **clear** / **CTRL+L** : clear the terminal
 - **exit** / **CTRL+D**: exit the shell session
 - ...and others!

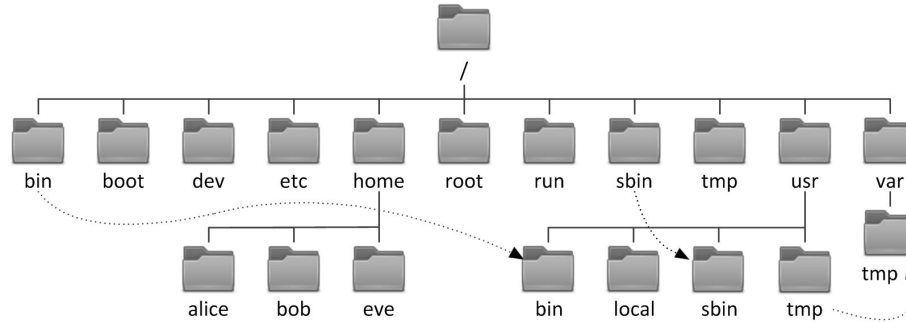
Navigating the filesystem

So where are we right now?

- At any time in the shell, you are located at one place in your **filesystem**
- Generally, commands can read/write/operate on files wherever you are
- We can find our **current working directory** with the command
 - `pwd`
- `pwd` stands for *print working directory*
 - Many bash commands are acronyms or abbreviations!
- Generally, a new terminal places you in your user's home directory (`~`)
 - `/home/$USER` (Linux) or `/Users/$USER` (macOS)

Navigating files and directories

- The **filesystem** is how the OS manages files and directories (folders)
- Shell commands are especially powerful for filesystem handling operations
 - i.e., creating, inspecting, and manipulating files and directories
 - Probably the most common use case for a shell

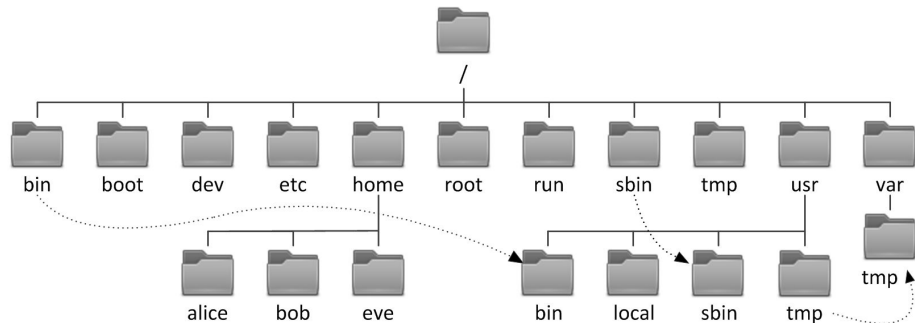


A directory tree 🌲

💡 macOS has slightly different directory names but the same principles

A quick tour of a Linux filesystem

- The top level directory, holding the entire filesystem, is the **root directory** (/)
- Some directories it contains:
 - `/home` is where personal user directories are kept
 - `/bin` contains built-in (*binary*) programs
 - `/usr` is where miscellaneous program files are stored
 - `/tmp` is for temporary files
- When not at the start of a path, `/` is a separator between directories



`/home/alice/Documents`

↑
root

↖ ↗
directory separator

Your home directory

- This is `/home/$USER` (or `/Users/$USER` on Mac)
 - `~` (tilde) is a shorthand
 - `$HOME` also stores the path to your home directory
- Where your user-specific files are
 - On a PC, your personal workspace directory
- Also commonly where configuration files are stored for tools you use
- On a remote shared filesystem (e.g., a lab server), you may have a project directory separate from your home directory for your work

What's inside a directory?

- To show the contents of our current working directory, we can use the command
 - `ls` (*list to screen*)
- To augment what and how much info `ls` displays, we can add **options** or **arguments**

General syntax of a shell command

Example: `ls -F interesting_files`

We have:

- A **command**: `ls`
- An **option** (flag, switch): `-F` (show the *type* of contents)
- An **argument**: `interesting_files`

Generally in that order

Options and arguments

- **Options** (a.k.a flags, switches) change the behavior of a command
 - They generally start with a `-` or `--`
 - They are case sensitive in most cases
 - `ls -r` and `ls -R` do different things
 - They can be combined (in any order)
 - e.g., `ls -lS` shows the contents of our working directory as a *list* (`-l`) that is *sorted in order of size* of contents (`-s`)
- **Arguments** (a.k.a parameters) tell the command what to operate on
 - Only sometimes optional (as with `ls`)
 - `ls` (list contents of *current working directory*) vs. `ls interesting_files`

How do I know what options are available?

- Many commands provide usage info via `--help` and/or `-h` options
 - `ls --help`
- `man` opens the **man(ual) page** for the command (not all programs have one)
 - `man ls`
- `whatis` provides a quick description of what a command does
 - `whatis ls`
- Availability of each type of command help will depend on (1) the command and (2) your OS

Changing directories

- We can also move between different filesystem locations using the *change directory* (`cd`) command, which takes a **path** argument
 - `cd flying_circus`

Changing directories

- Typing `cd` without any arguments takes you back to your home directory
 - So does: `cd ~`
 - Recall: `~` (tilde) is a shorthand for home directory (`/home/$USER`)
- `-` (hyphen) is a shorthand for **previous working directory**
 - `cd -` takes you back to the last directory you were in

💡 A trailing `/` in a path is an optional convention for directories

Absolute and relative paths

- You can `cd` multiple directory levels at once by stringing them together with `/`
- Changing directories can be done with either absolute or relative paths
- **Absolute paths** are defined in reference to the root directory
 - Begin with `/`
 - Most precise, but also very verbose to type
 - e.g., `/home/adai/shell-course/flying_circus`
- **Relative paths** are defined in reference to the current working directory
 - Do not begin with `/`
 - e.g., `shell-course/flying_circus`, `./shell-course/flying_circus`, `../scratch`

Navigating using relative paths

- `..` is a shortcut for “the directory containing the current one,” the **parent directory**
 - This can be repeated (`cd ../ ../ ..`)
- `.` is a shortcut for the current working directory
- `ls -a` reveals these shortcuts in a directory
 - `-a`: *show all contents, including special and hidden files/directories*

Hidden files

- `ls -a` also reveals other **hidden** files/directories
- These are prefixed with `.` (dotfiles) and can be either files or directories
- Hidden files often store configuration settings or version control records
- They are not displayed by default to reduce clutter

Summary - Basic navigation

- `echo` prints text or variable values to the terminal
- `pwd` prints the current working directory
- `ls` lists info about a specified path, or the current working directory contents (default)
- `cd` changes the current working directory to the specified path, or `~` by default
- Absolute paths specify locations starting from the filesystem root `/`
- Relative paths specify locations starting from the current working directory
- `.` means “current directory,” `..` means “parent of current directory”

Working with files and directories

Creating directories

- Let's create a directory to store an imaginary dataset:
 - `mkdir fake_dataset`
- This will add a directory named `fake_dataset` inside your current working directory (verify with `ls -F`)
- Good naming conventions for files and folders:
 - DO NOT use `s p a c e s` of any kind
 - DO stick to letters, numbers, `-`, `_`, and `.`
 - Avoid other special characters, e.g. `~!@#$%^&*()`
 - DO NOT begin the name with `-`

Creating files

- To create a new file in a directory, you can
 - Open a new file using a terminal text editor (`nano`, `vim`, etc.)
 - Open a new file in VS Code from the command line (`code`)
 - `touch` a new, empty file into existence
- You can print the contents of a file using `cat` (*concatenate*)
- Generally, the shell works well with *plaintext* files
 - In contrast to *binary* files (.docx, .xlsx, .png, etc.)
 - Not human-readable without specialized programs



Plaintext files don't need to end with .txt (or have any extension at all)

nano

- `nano` is a user-friendly text editor that runs inside the terminal window
 - Useful on remote systems where you cannot easily open GUIs
- Uses the `CTRL` (^) and `ALT` (M) keys to make changes
 - Available commands are shown at the bottom of the editor window
 - `^O` : press and hold `CTRL`, then press the letter `O` key
 - `M-U` : press and hold `ALT`, then press the letter `U` key
 - Save your changes with `^O`, then press `ENTER` to confirm the filename

Creating files - redirection basics

- We may want to store text (output) that programs print to the terminal
- We can capture this output by *redirecting* it to a file using **redirection operators**
 - `echo "Text I want to save" > LOG.txt`
 - `>` redirects output text to a file and **overwrites** (replace) whatever is in the file
 - `>>` redirects output text to a file and **appends** to the file if it exists
- Commonly used for creating logs

Moving files and directories

- The `mv` command can be used to *move* files or whole directories
 - `mv <source-path> <destination-path>`
- `<destination-path>` (final argument) can be a destination directory or a rename target
 - Renaming a file/directory = *moving* it to a new name in its current location
- You can move multiple items at once, as long as the final argument is an existing directory
 - `mv file1.txt file2.txt folder`

Pattern matching file and directory names

- We can use **wildcard** characters to match names of files and directories based on specific patterns
 - Referred to as **globbing**
- ***** will match any character zero or more times
 - ***.txt** will match **a.txt** and **any.txt**
- **?** will match any character exactly once
 - **? .txt** will match **a.txt** but not **any.txt**
- So, we can move multiple files at once without typing each filename

Beware of silent overwriting!

- `mv` will silently overwrite files at the destination if a file with the same name already exists!
 - This can not be undone!
- Use `ls` and other tools to check what's in a directory before moving/renaming
- Consider using `mv -i` for *interactive* moving (warn before overwriting)

Copying files and directories

- `cp` is a lot like `move`, but for *copying* files and folders
 - `cp <source-path> <destination-path>`
- `<destination-path>` can be a destination directory or the new copy name
- All the same wildcard rules apply to `cp`
- Copying a directory requires the `-r` (*recursive*) flag
- **Warning:** `cp` will overwrite existing files or directories with the same name at the destination! (See also `cp -i`)
 - `rsync` will compare the source/target and only transfer what is needed - even between machines
 - `rsync -av <source> <destination>`

FYI: Linking files and directories

- `ln -s` (*symbolic link*) is similar to copy, but a copy isn't actually made
 - `ln -s <target-path> <shortcut-path>`
- It creates a named shortcut or “link” that points to a target file or directory
- Opening the shortcut opens the linked file/directory - there is no duplicate!
 - Changes to the “link” will be reflected in the source (target)
- Useful for frequently-accessed locations, or when working with shared data

Removing files and directories

- `rm` is the command to delete files and directories
- Removing a directory requires the recursive flag (`rm -r`)
-  You **cannot recover** files that have been deleted with `rm`! 
 - The shell **does not have a trash/recycle bin**
 - Consider: `rm -i` to be prompted for each item that will be deleted
- There is also `rmdir`, which only removes empty directories

Summary - Basic directory and file handling

- `mkdir <new-directory>` makes a directory
- `touch <file>` pokes a file or creates a new, empty one
- `mv <source> <destination>` moves (renames) a file/directory
- `cp <source> <destination>` copies a file
- `>`, `>>` redirects output
- `rm <path>` removes (delete) a file
- `rmdir <empty-directory>` removes an empty directory
- `nano` is a basic terminal text editor

Finding things using the shell

Finding files

- A common problem is finding specific files in a deeply nested filesystem (many subdirectories inside subdirectories)
- To display **all** files inside a directory (including subdirectories), you can use:
 - `ls -R` (note the capital **R**!)
 - `tree` (*may not be installed*)
- To search for filenames with a specific pattern anywhere inside a directory (traversing the **whole directory tree**), use `find`
 - `find . -name "my_*`
 - You can use options to further filter the results

Finding things inside of files

- To search across *contents* of files instead of their filenames, we can use `grep` (*globally search for a regular expression and print the matching lines*)
 - `grep "Bash" helloworld.txt`
- This will print **lines** that match the pattern **inside** the matching text file(s)
 - `grep -r "Bash" *.txt`
- Example: what if we wanted to quickly find mentions of “rabbit” (case-insensitive) inside any file in `flying_circus/`?
 - Note that a filename `dangerous_rabbits.txt` would not be treated as a match

Exploring contents of a file

- There are several useful ways to explore text files from the command line:
 - `head` : print the first `-n` lines (default 10)
 - `tail` : print the last `-n` lines (default 10)
 - `wc` : print the *word count*; `-l` counts the *number of lines*
 - `wc -l` is very useful for files where each line corresponds to a subject/sample
 - `sort` : sort lines alphanumerically
 - `cat` : print the whole file

Passing information - Pipes |

- A strength of the shell is that you can connect the output of one command to the input of another (and then that output into another, etc.)
 - Combining functionality of multiple commands into a **pipeline**
- This is done with the **pipe** character: |
 - `command1 -flags arguments | command2 -flags arguments`
 - “Pass the output of command1 directly as the input to command2”

Passing information - Pipes |

- Example: What if we want to find the **first 5 occurrences** of “rabbit” in `flying_circus/the_holy_grail.txt`?
 - Note: `grep` has options for achieving the same result without a pipe!
 - In general, there are multiple ways to do the same thing in a shell
- You can pipe as many commands together as needed, e.g.:
 - `ls -lF | head`
 - `find . -type f -name "sub-*_output.txt" | sort | tail -n 5`

FYI: Pipes and `xargs` (advanced)

- With pipes, you have the option to:
 - work with the entire output of the pipe (previous examples)
 - when a pipeline outputs a list, use `xargs` to work with each output individually
- Compare:
 - `find . -type f -name "sub-*_output.txt" | sort | tail -n 5`
 - `find . -type f -name "sub-*_output.txt" | sort | xargs -n 1 tail -n 5`

Summary - Finding things with the shell

- `tree` shows you the structure of a directory, including all nested files/subdirectories
- `find` is used to search for files/directories based on name patterns (and other metadata)
- `grep` is used to search within file(s) for occurrences of a given string (or even complex regular expression)
- We can quickly explore a file without opening it, using tools like `head`, `tail`, and `wc`
- `cat` prints the contents of a file
- `sort` sorts the order of its input lines
- Pipes `|` allows us to connect the output of one command to the input of another command

Scripting basics

Shell scripts

- A principal power of the shell is the ability to write sequences of commands as reusable scripts
- This is helpful if you want to:
 - Run the exact same commands or process again
 - Document the commands and parameters used to produce an output
 - Generalize a working analysis to new inputs or parameters
 - Share a set of commands with other users

Anatomy of a shell script

- A plaintext file containing shell commands
- Conventionally has the extension `.sh`
- At the top of the file is the *shebang* line
 - `#!/bin/bash`
 - Tells the system what shell program to use to interpret (execute) the script, e.g., bash
- `#` anywhere else denotes a *comment*, text that will not be executed

Running a shell script

- To run (execute) a shell script, simply specify its path:
 - `./run_me.sh`
 - Include `./` to run a script in the current directory
- However, for this to work, the script must also have the correct **file permissions!**

File permissions

- File permissions control what actions can be taken on a file
 - *Read (r)* - view contents
 - *Write (w)* - modify contents
 - *Execute (x)* - run the program
- Different users may have different levels of access to a file
- Shell scripts must have **execute permissions** to be run
- You can change file permissions with **chmod** (*change mode*)
 - `chmod +x run_me_too.sh`
 - Same as: `chmod ugo+x run_me_too.sh`

```

rwxrw-r--
  {   {   {
  user group others

```

💡 `ls -F` lists executable files with `*`

How does the shell know where to find commands?

- Why don't we need to specify the command path when we run `cd` or `ls`?
- By default, the shell looks for executable programs in all the places it's aware of, which are stored in the variable `$PATH`
 - An environment variable set by system- and user-level startup files
 - Different directories are delineated by `:`
- Adding a directory to `$PATH` makes the executables inside available to you by name
- Otherwise, you must specify the relative or absolute path to a script to run it

How does the shell know where to find commands?

- Directories in `$PATH` are searched in order - the first matching program is run
- Use `which` to find the path of a program you are calling (within `$PATH`)
 - Recall from course setup: `which python`
 - Very helpful if you have multiple versions of a tool with the same name

Modifying \$PATH on startup

- `~/.bashrc` is a configuration file that runs on startup of each new bash session
 - On macOS, `~/.bash_profile` serves the same purpose
- It is used to make changes to `$PATH` that persist across your sessions
- Installing new tools may modify your `$PATH`
- This tells the shell to search additional directories for programs



`~/.bashrc` also does other things, like let you define aliases (shorthands) for commands

Creating your own shell variables - \$



\$PATH is just another variable

- Shell variables are set with `=` (with no spaces around it)
 - `session_num=2`
 - `DATASET=HCP`
- Use `$` to reference the **value** of a variable
 - `echo $session_num`
- (Recommended) use `{ }` to explicitly delineate the variable name
 - `echo session${session_num}_results`
- Variable names are case-sensitive. By convention, they are:
 - `ALL_CAPS` if they are **environment variables** or don't change once set (constants)
 - `lower_case` if they are only used within the local scope (e.g., **shell variables**)

Shell vs. environment variables

- **(Regular) shell variables** are only available in the current shell instance (locally scoped)
 - not shared with any programs executed from this instance
- **Environment variables** are available to any program you execute in your shell
- A new variable starts off as a shell variable; to “promote” it to an environment variable, you must `export` it
 - `export DATASET`
 - `export SITE=Washington`
 - Note that we export the variable itself, *not* its value (no `$`)
 - To “demote” an environment variable: `export -n MY_VAR`
- To remove a variable: `unset MY_VAR`

Environment variables are passed down to programs

Example:

Let's set some variables for `interesting_files/i_can_see_variables.sh`

Modifying environment variables

- You can edit any environment variable, including `$PATH`, using `export`
 - `export PATH=/opt/newtool:$PATH`
 - This only changes the variable for your current shell session
- To set/change (export) an environment variable “permanently,” add the line to your `~./bashrc`
 - **WARNING:** Editing `~./bashrc` should be done rarely and with extreme caution!

Summary - Scripting basics

- Shell scripts contain commands to be executed in a sequence
- The *shebang* (e.g. `#!/bin/bash`) in a script declares what shell should run it
- Use `chmod` to change file permissions (e.g., grant execute permissions)
- The shell will look for programs in directories in the `$PATH` environment variable
- To run a shell script or command not in `$PATH`, specify the path to it
- `~/.bashrc` is an editable startup configuration file that can set `$PATH`
- In the shell, you can create and edit variables: environment variables and shell variables
- `$` is used to reference variable *values* (e.g., `$VAR` vs. `VAR`)
- `export` “promotes” a shell variable to an environment variable

Questions?

High Performance Computing (HPC) clusters

A brief introduction

Working on an HPC cluster

- These are shared (multi-user) rather than “personal” systems
- You don’t interact with them directly, but over a network connection in a shell (`ssh`)
- Typical interaction for an analysis:
 - Transferring data and results
 - Managing your analysis “jobs”
- Most of the computing work is non-interactive
 - Run on **compute nodes** (each like an individual computer)
 - The cluster is a network of such nodes
- Visualization is performed locally, not on the HPC
- They almost exclusively run Linux



Advantages of a HPC over a PC

- Able to handle large datasets more efficiently
- The option to “**Scale Out**” or “**Scale Up**” an analysis
 - **Scale Out:** analyze independent parts of a dataset simultaneously
 - Independent permutations / cross-validations / simulations
 - Different subjects through the same (pre)processing tasks
 - “Embarrassingly parallel”
 - High throughput computing
 - **Scale Up:** create a larger single instance of computing resources to run a larger model, estimation, or analysis

Common commands for working with HPC jobs

- `sbatch`
 - Submit a job to the scheduler.
- `squeue`
 - Monitor the status of jobs on the cluster.
- `scancel`
 - Cancel a job that hasn't completed yet.
- `sacct`
 - Show a summary of the jobs you recently submitted.
- `srun`
 - Start an interactive job to test that your job will work correctly.
 - `srun --nodes=1 --ntasks-per-node=1 --time=01:00:00 --pty bash -i`



There are many great resources online, like the Digital Research Alliance of Canada (formerly Compute Canada) docs: https://docs.alliancecan.ca/wiki/Getting_started